CBDSALG MCO1 Specifications - Expression Evaluation (Stack and Queue Application) Term 3 AY 2025 - 2026

Krystel Yvonne Collado, Fatima Joy Orolfo, Ayessa Tesorero
Laguna Boulevard, LTI Spineroad
Laguna, Biñan City, Philippines
krystel_collado@dlsu.edu.ph, fatima_orolfo@dlsu.edu.ph, ayessa_tesorero@dlsu.edu.ph

ABSTRACT

This document details the output for MCO1 of Group 5, a software project that utilizes the lessons of the course to implement a calculator using stacks and queues as key data structures. The initial coding of the program was done in a main C file uploaded and updated in a Github repository for ease of collaboration and version control. Multiple functions were developed for the infix-to-postfix conversion first, including (1) a function to check if the stack is empty or full, (2) a function to check the value at the top of the stack, (3) and the push and pop functions. An array was initially used however, it was decided it would be easier to use a linked list using data structures, ending with most of the code being scrapped and redone. Using the linked list, the stack and queue data structures were implemented and used for the functions of the calculator. The shunting-yard algorithm was used as a base for the infix-to-postfix function. The evaluation of the postfix utilized token queues to traverse the expression from left to right. For the testing process, we came up with our own test cases and an actual infix to postfix calculator, CalCont, was utilized to verify if the outputs of our program were correct. Any problems with the program were dealt with during the testing process. The limitations of the program were revealed after the testing process, including the ability to only be able to evaluate integers as well as the inability to parse through expressions that have a space in between them.

STACK AND QUEUE IMPLEMENTATION

To save time and make it easier for us to collaborate, all the coding was initially done in the main C file. After all the functions were done, that's when we started separating the functions into different C files. This is so that we don't have to keep pulling the updates from Github for multiple C files whenever someone commits or pushes.

We first decided to look for references online on how to create linked lists in C. We found a sample code and analyzed it to understand how it worked. After that, we modified it to fit what we need to use it for, namely the infix to postfix conversion. The sample code used hardcoded values that were manually assigned. On the other hand, our code had to allow the users to input an infix expression with at most 256 characters during runtime.

Stack

For the stack, we created a linked list made using a struct called Node. Each Node struct stores a token, either an operand or an operator, and a pointer to the next node. The push() function was also made to help insert a new node at the end of the linked list. First, we allocated a memory for a new node using malloc. The push function pushes the value into the stack. It works by first checking if the stack is empty. If it is, the new node becomes the head. If it's not, the new node is linked to the last node, and then the new value is set as the tail.

On the other hand, the pop() function removes the value at the top of the stack. First, it checks if the list is empty. If it is, it sets the result to be empty. Otherwise, we make two struct Nodes: one which acts as an indicator on which position we are currently at in the list, named as temp, and another which indicates the node before the current position, named prev. While the next node after temp is not equal to NULL, meaning we're not at the end yet, we set the previous node (prev) as (temp) or the value after it, and then temp is set as the next value or node after it. Then, we use strcpy to transfer the data or the new

values of the node to the result array, which stores the new linked list after popping. If prev is NULL, it means there is only one node left in the stack, so when you pop it, the stack becomes empty. Otherwise, we just pop the last node. After popping, the popped item is erased from the linked list by removing the link so it can't be accessed anymore.

Queue

The queue utilizes a linked list similar to the stack. The queue structure sets the head and the tail of the queue. To ensure the queue works without any errors, the initQueue function initializes the queue to NULL before doing anything with the infix. The enqueue function works similarly to the push function however, it utilizes the first-in, first-out policy, unlike the stacks. Moreover, the dequeue function first checks if the queue is empty, otherwise if the queue is not empty, a temp struct is created that's equal to the head of the queue. Next, the result copies the contents of temp as the current head moves onto the next operand/operator. If the head is equal to NULL, the tail is also set to NULL.

INFIX TO POSTFIX IMPLEMENTATION

For the infix to postfix implementation, we first initialized an empty stack to temporarily store the operators during conversion.

The precedence function is used to easily identify the order when to pop or push the said operand/operator to the Postfix.

After all the functions were made, we coded the algorithm for the infix to postfix conversion. The function will only continue if the head is not equal to NULL. This is to ensure that the program will continue as long as there are still tokens left to be read in the infix expression. While the head is not equal to NULL, the token is first initialized. Tokens are broken down into token and multiToken in the function, holding the current digits for operand and operator. However, multitoken can have up to the MAX (256) number of characters.

The function will first look for any multi-digit operands, or any operand that has more than one digit. As long as the while loop satisfies the conditions of the head not being equal to NULL and the current digit is an operand, the multitoken will continue adding digits to its array. Once the loop hits an operator, the final index of i closes the multiToken by inputting '\0'. This is then queued using the enqueue function into the postQueue and appended.

After the function finds the full number and stores it into the queue, the program will then check if the operator is a multi-character operator. If the value after the current head is not equal to NULL, curr is equal to the current head and next is equal to the character after head. The program will check if any of the multi-character operators is the current operator and store the results of curr and next into tokens, setting the next head to be the value after next. If the operator wasn't a multi-character variant, the current head is tokened and set to the character after the current head.

The program will then check for any parenthesis that may be present in the infix. First, if an open parenthesis "(" is the current token, the push function is used on the current token. Otherwise, if a closed parentheses ")" is the current token, it enters a while loop while the stack is not empty and while the top of the stack is not equal to an open parenthesis "(", the stack is repeatedly popped and queued into the postfix expression.

Otherwise, if the operator is regular, the program first enters a loop to check for precedence. While the stack is not empty, the top is not an open parenthesis, and the operator on the stack has a higher or equal precedence than the current token, it will continue to pop the stack and queue it into the postfix expression. Afterwards, the current token is pushed onto the stack.

Finally, once the head reaches NULL and the expression is fully read, a final loop runs to pop any of the remaining operators left in the stack and queues them into the final postfix expression.

POSTFIX EXPRESSION EVALUATION IMPLEMENTATION

For the postfix evaluation, we first initialize an empty stack that will store the operands and intermediate result. The postfix expression is processed from left to right by traversing each token in the queue.

If the current token is an operand, it is pushed onto the stack. If the current token is an operator, the required first and second operands are popped from the stack, converted from string to integers using the atoi function, and the operation is performed.

The Logical NOT (!) operator only requires one operand (unary operator), while operators such as arithmetic, relational, equality, and logical operators require two operands (binary operator). For binary operations, the second operand is popped first, followed by the first operand to maintain the correct order of operations.

After computing the result of an operation, the result is converted back into a string and pushed back onto the stack, allowing the result to be used again as an operand for another operation in the postfix expression.

The process continues until all of the tokens in the queue have been evaluated. If a division by zero is encountered, the expression is marked invalid and the evaluation stops.

Once all of the tokens have been evaluated, the final value is popped, then converted into an integer, and returned as a result of the postfix expression.

TESTING PROCESS

We thought of our own test cases to use, but we did use a site called "CalCont" to determine their expected outputs to help us verify if our program runs correctly and produces the right results. We came up with test cases starting by using single operators, then gradually came up with more and more complicated expressions. This is so we can make sure each one works without a problem and pinpoint where the problems occur easier. First, we tested infix expressions with arithmetic operators (+, -, *, %, ^), followed by relational operators (>, <, >=, <=, !=, ==) , then logical operators (!, &&, ||), and parenthesis grouping.

Some of the notable problems we encountered include:
(1) When the infix expression includes a number divided by 0, the program still evaluates the rest of the equation and outputs the result instead of printing an error statement and ending the evaluation.
(2) Inputs starting with "!" followed by another operator outputs the wrong postfix expression and evaluation.

The changes we made include:
(1) Revision of the evaluatePost function by adding a variable called valid to determine if the expressions can be solved. Then, we passed that value back to the main function, so that the program doesn't print the result anymore and prints an error message instead.
(2) After searching, we realized that the associativity of the "!" operator is right to left, unlike the other operators. So, we added a bool variable that checks whether an operator is left or right associative and added them in the while loop at the end of the infixtoPost function. If the operator is left-associative, operators with greater or equal precedence are popped. On the other hand, if the operator is right-associative, only operators with greater precedence are popped.

LIMITATIONS

The program can only evaluate integers that have no spaces in between them for simplicity of the program as specified in the specifications.

REFERENCES

[1] Programiz. (n.d.). *C Precedence and associativity of operators: definition and examples.* Programiz. https://www.programiz.com/c-programming/precedence-associativity-operators

[2] GeeksforGeeks. (2025, July 23). *How to create a linked list in C?* GeeksforGeeks. https://www.geeksforgeeks.org/c/how-to-create-a-linked-list-in-c/

[3] GeeksforGeeks. (2026, April 9). *Dynamic Memory allocation in C.* GeeksforGeeks. https://www.geeksforgeeks.org/c/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/

[4] GeeksforGeeks (2025, October 13). *atoi() Function in C.* GeeksforGeeks. https://www.geeksforgeeks.org/c/c-atoi-function/

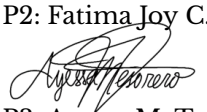
[5] GeeksforGeeks. (2025, July 23). *How to convert an integer to a string in C?* GeeksforGeeks.

https://www.geeksforgeeks.org/c/how-to-convert-an-integer-to-a-string-in-c/
[6] Khamkar, A. (n.d.). Infix to postfix calculator. https://www.calcont.in/Conversion/infix_to_postfix
[7] Shunting Yard Algorithm | Brilliant Math & Science Wiki. (n.d.). https://brilliant.org/wiki/shunting-yard-algorithm/

Appendix A: Record of Contribution
Group Members:

- P1: Krystel Yvonne Collado

- P2: Fatima Joy C. Orolfo



- P3: Ayessa M. Tesorero

Activity	Contribution of each member		
	P1	P2	P3
Stack Implementation	10.0	40.0	50.0
Queue Implementation	100.0	0.0	0.0
Infix to Postfix Conversion	80.0	10.0	10.0
Postfix Evaluation	10.0	45.0	45.0
Testing	15.0	50.0	35.0
Documentation	33.3	33.3	33.3
Raw Total	248 .0	178.3	173.3
TOTAL	**41.36%**	**29.74%**	**28.90%**

Appendix B: AI Use Declaration
Table 1. AI Use Declaration

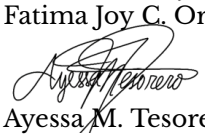
Task	N	S	R	G
Understanding Specifications				
Understanding Algorithm or References				
Stack Implementation				
Queue Implementation				
Expression Conversion				
Expression Evaluation				
Documentation				

For each of the following tasks, identify the degree of AI use
among the following:
• *N* - None - No AI was used for this task
• *S* - Scaffolding - AI was used to build the foundation for this task, e.g., brainstorming, solution ideation
• *R* - Refinement - AI was used to review, provide feedback, and revise outputs for this task
• *G* - Generation - AI was used to generate portions of the output for this task

Statement of Accountability:
We affirm that all AI-assisted contributions have been critically examined by the project group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full accountability for all content in this project and documentation.

Krystel Yvonne Collado

Fatima Joy C. Orolfo



Ayessa M. Tesorero